

SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

Project: Web Based ML Model Evaluator

Version: 1.0

Authors: Vishal, Vivek, Vedant, Akshith

SRN(s): PES2UG24CS592

Section: 5J

Date: 14-09-2026

Status: Draft

Table of Contents

- 1. Introduction
- 2. Document Overview
- 3. Architecture
 - 3.1 Goals and Constraints
 - 3.2 Stakeholders and Concerns
 - 3.3 Component Diagram
 - 3.4 Component Descriptions
 - 3.5 Architecture Pattern and Rationale
 - 3.6 Technology Stack and Data Stores
 - 3.7 Risks and Mitigations
 - 3.8 Traceability to Requirements
 - 3.9 Security Architecture
- 4. Design
 - 4.1 Design Overview
 - 4.2 UML Sequence Diagrams
 - 4.3 API Design
 - 4.4 Error Handling, Logging and Monitoring
 - 4.5 UX Design
 - 4.6 Open Issues and Next Steps
- 5. Appendices

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification (SADS) defines the proposed architecture, major components, interfaces, data stores, security controls, design flows, and traceability for the Web Based ML Model Evaluator. It translates the project SRS into an implementable modular web application design.

1.2 Scope

The architecture covers dataset upload and validation, target and feature configuration, built-in algorithm selection, train-test split configuration, server-side model training and evaluation, metric visualization, PDF/CSV report export, minimal authentication, and per-user evaluation history. The design does not include user-supplied pretrained models, distributed or large-scale training, or production-grade MLOps pipelines.

1.3 Audience

The intended readers are project developers, QA evaluators, the course instructor/TA, security reviewers, maintainers, and end users who need to understand the system structure and behavior.

1.4 Definitions

Term	Definition
ML	Machine Learning
CSV	Comma-Separated Values
API	Application Programming Interface
UI	User Interface
F1-score	Harmonic mean of precision and recall
ROC-AUC	Receiver Operating Characteristic—Area Under Curve
JWT	JSON Web Token
RTM	Requirements Traceability Matrix
SADS	Software Architecture and Design Specification

2. Document Overview

2.1 How to Use This Document

Sections 3 and 4 provide the architectural and detailed design deliverables. The component diagram explains static structure; the sequence diagrams explain runtime interactions; the API section defines service boundaries; and the traceability section connects SRS requirements to design elements and verification activities.

2.2 Related Documents

- Software Requirements Specification (SRS), Web Based ML Model Evaluator, Version 1.0, dated 07-09-2026.
- Software Test Plan and test-case specifications, to be prepared or maintained by the project team.
- Requirements Traceability Matrix (RTM) included in the project SRS and extended in this document.

2.3 Product Perspective

The system is a standalone browser-based application. A responsive frontend communicates with a backend REST API over HTTPS. The backend validates uploaded CSV files, coordinates model evaluation using a standard ML library such as scikit-learn, stores user accounts and evaluation history in a relational database, and returns metrics, visualizations, and exportable reports.

3. Architecture

3.1 Goals and Constraints

Architecture goals:

- Clear separation of presentation, API, business logic, ML evaluation, persistence, and reporting responsibilities.
- Support guest evaluation while restricting history persistence to authenticated users.
- Provide deterministic, testable evaluation workflows and standardized error responses.
- Allow new built-in algorithms to be added with changes confined primarily to the evaluation-engine module.
- Protect credentials, uploaded datasets, evaluation results, and history through authentication and user scoping.

Constraints:

- Initial input format is CSV only, with a maximum upload size of approximately 10 MB.
- Only built-in algorithms are supported: Logistic Regression, Decision Tree, Random Forest, SVM, and K-Nearest Neighbors.
- The initial system is intended for datasets up to approximately 10,000 rows and 50 columns for the stated performance target.
- Internet connectivity and a modern browser are required; no offline mode is specified.
- Production deployment must use HTTPS/TLS and secure password hashing.

3.2 Stakeholders and Concerns

Stakeholder	Primary Concerns	Architectural Response
End users / students	Simple workflow, understandable metrics, quick feedback	Step-based UI: Upload → Configure → Evaluate → Results; charts and summary tables.
Registered users	Privacy and access to saved evaluations	Authenticated history endpoints and strict owner-based data filtering.
Developers	Modularity and ease of extension	Layered modular backend and algorithm adapter/registry.
QA evaluators	Repeatability, measurable acceptance criteria	Stable API contracts, unit tests for metrics, and RTM-linked test cases.
Instructor / reviewer	Traceability, security, maintainability	Documented design decisions, security controls, and requirement mapping.
System administrator (optional)	Usage visibility and diagnostics	Optional admin-only usage statistics and log views.

3.3 Component (UML) Diagram

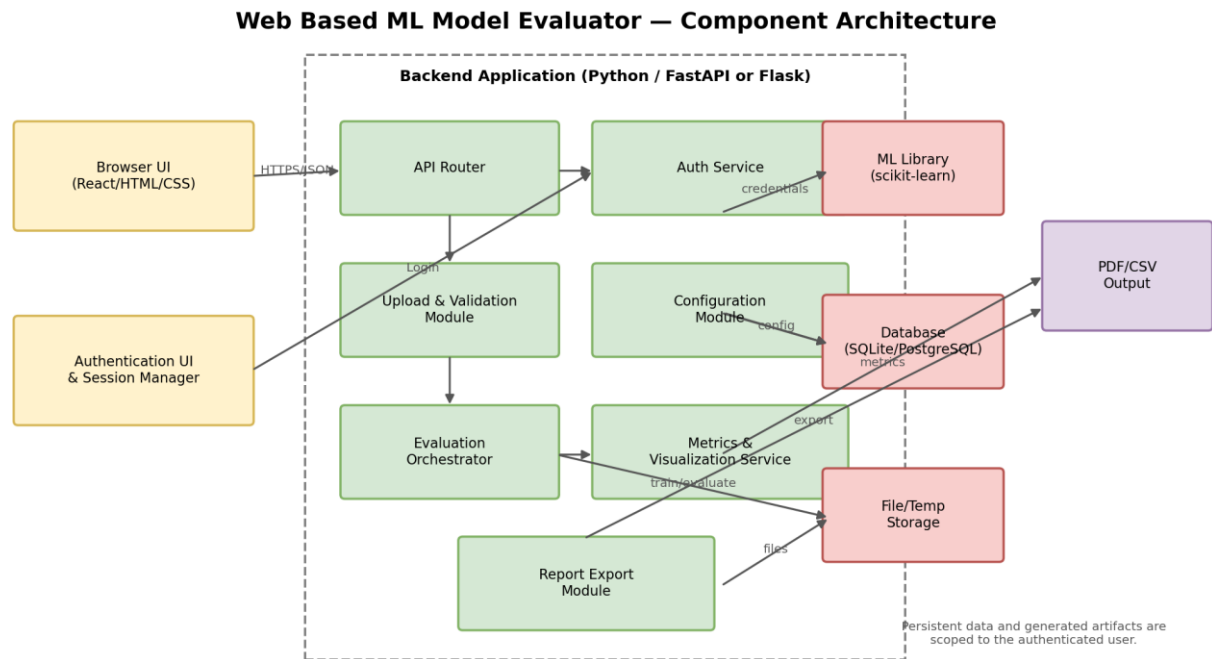


Figure 1. Component architecture using a draw.io-style UML/component visual.

The architecture is organized around a browser client, a backend application boundary, ML and persistence dependencies, and report outputs. The API Router is the single entry point for frontend requests. Internal modules separate authentication, upload validation, configuration, evaluation orchestration, metrics/visualization, and report generation.

3.4 Component Descriptions

Component	Responsibilities
Browser UI	Provides the responsive dashboard and the Upload → Configure → Evaluate → Results flow. It collects user input, displays validation errors, renders charts/tables, and initiates report downloads.
Authentication UI / Session Manager	Handles registration, login, logout, token/session storage on the client side, and authenticated request headers. It does not implement password hashing.
API Router	Exposes REST endpoints, validates request structure, authenticates protected requests, maps exceptions to HTTP responses, and delegates work to internal modules.
Auth Service	Registers users, verifies credentials, hashes passwords using a strong one-way algorithm such as bcrypt, issues or invalidates JWT/session tokens, and applies login/register rate limiting.
Upload & Validation Module	Checks file extension/MIME type, size, CSV structure, column names, parseability, and basic data quality. It generates a preview and reports missing-value handling.
Configuration Module	Validates target-column and feature-column selections, algorithm selections, and train-test split ratios. It creates a normalized evaluation configuration.
Evaluation Orchestrator	Coordinates preprocessing, train-test splitting, algorithm execution, metric computation, and aggregation of per-

	algorithm results.
ML Library Adapter	Wraps scikit-learn estimators behind a common interface so algorithms can be selected by identifier and added without changing the API layer.
Metrics & Visualization Service	Computes accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC where applicable. It prepares chart and table data for the frontend.
History Repository / Database	Stores user accounts, evaluation-run metadata, selected algorithms, configuration, metrics, timestamps, and references to generated artifacts.
Report Export Module	Builds PDF reports and raw CSV metric exports containing the run's metrics and displayed result summaries.
File/Temp Storage	Holds validated uploads and temporary/generated artifacts with controlled lifecycle and user scoping.

3.5 Chosen Architecture Pattern and Rationale

A layered modular monolith is selected for the initial version. The presentation layer, API layer, application/service layer, ML evaluation layer, persistence layer, and reporting layer have explicit responsibilities.

- Why selected: the project is a standalone application with moderate scope, limited data volume, and no requirement for independent deployment of services.
- Benefits: simpler deployment, lower operational overhead, easier debugging, clear module boundaries, and straightforward transaction/history management.
- Microservices are not selected for the initial version because service discovery, distributed tracing, independent deployment, and inter-service reliability would add complexity without a demonstrated need.
- The evaluation engine remains modular internally, allowing a future extraction into a worker service if concurrency or dataset size increases.

3.6 Technology Stack and Data Stores

Layer	Recommended Technology	Purpose
Frontend	React or equivalent HTML/CSS/JavaScript frontend	Responsive dashboard, forms, charts, result tables, and authentication state.
Backend API	Python with FastAPI or Flask; REST and JSON	Request routing, validation, authentication, orchestration, and error responses.
ML computation	pandas, NumPy, scikit-learn	CSV parsing, preprocessing, model training, prediction, and metric calculation.
Database	SQLite for development; PostgreSQL for deployment	Users, evaluation history, configurations, metrics, timestamps, and artifact metadata.
Visualization	Frontend chart library or server-prepared chart data	Metric comparison bar charts, confusion-matrix heatmaps/grids, and ranking tables.
Reporting	PDF generation library and CSV serialization	Export of evaluation results, metrics, charts/tables or their report equivalents.
Transport/security	HTTPS/TLS 1.2+; JWT or secure session cookies	Confidential client-server communication and authenticated requests.
Deployment	Standard server/container environment	Reproducible runtime without specialized hardware.

Data model overview:

- User(user_id, username/email, password_hash, created_at, role)
- EvaluationRun(run_id, user_id nullable for guest scope, dataset_name, timestamp, target_column, feature_columns, split_ratio, status)

- AlgorithmResult(result_id, run_id, algorithm_name, accuracy, precision, recall, f1_score, roc_auc, confusion_matrix, serialized_metrics)
- Artifact(artifact_id, run_id, artifact_type, storage_reference, created_at)

3.7 Risks and Mitigations

Risk	Impact	Mitigation
Malformed or malicious upload	Crashes, resource exhaustion, or data leakage	Enforce size/type limits, parse safely, validate schema, sanitize input, isolate temporary files, and return controlled errors.
Long-running evaluation	Poor responsiveness or request timeout	Set dataset limits, use bounded execution, provide progress/status if needed, and queue requests beyond concurrency capacity.
Metric misuse	Misleading comparisons	Display metric applicability, identify binary-classification ROC-AUC conditions, and test against known datasets.
Unauthorized history access	Privacy breach	Require valid authentication and filter every history query by authenticated user ID.
Credential compromise	Account takeover	Use bcrypt/strong hashing, generic login errors, HTTPS, secure token handling, and rate limiting.
Algorithm implementation coupling	Difficult maintenance	Use a registry/adaptor interface for built-in estimators and isolate algorithm-specific configuration.
Database or storage failure	Loss of history or artifacts	Use transactions, consistent status fields, cleanup policies, backups in deployment, and clear failure responses.

3.8 Traceability to Requirements

SRS Requirement	Architecture Element	Design Responsibility	Verification
MLE-F-001 to MLE-F-005	Auth Service; Session Manager	Registration, hashing, login, logout, invalid-credential handling	TC-001 to TC-005
MLE-F-006 to MLE-F-009	Upload & Validation Module	CSV upload, type/size checks, parsing, preview	TC-006 to TC-009
MLE-F-010 to MLE-F-012	Configuration Module	Target/features and missing-value strategy	TC-010 to TC-012
MLE-F-013 to MLE-F-015	Configuration Module; ML Library Adapter	Algorithm list, multi-selection, split validation	TC-013 to TC-015
MLE-F-016 to MLE-F-019	Evaluation Orchestrator; Metrics Service	Training, evaluation, metrics, confusion matrix, ROC-AUC	TC-016 to TC-019
MLE-F-020 to MLE-F-022	Metrics & Visualization Service; Frontend	Charts, heatmaps/grids, ranking table	TC-020 to TC-022
MLE-F-023 to MLE-F-024	Report Export Module	PDF and CSV export	TC-023 to TC-024
MLE-F-025 to MLE-F-027	History Repository; Auth Service	Save, list, and delete user-scoped history	TC-025 to TC-027
MLE-NF-001 to MLE-NF-006	Evaluation Engine; Backend; Frontend	Performance, usability, concurrency, reliability, portability, modularity	TC-N01 to TC-N06
MLE-SEC-001 to MLE-SEC-005	Auth, Upload, API, Infrastructure	Hashing, sanitization, authorization, TLS, rate limiting	TC-SEC-001 to TC-SEC-005

3.9 Security Architecture

The security design follows the SRS security objectives and applies defense-in-depth at the client, API, application, storage, and deployment layers.

- Authentication: registration and login issue a JWT or secure session token; protected history endpoints reject unauthenticated requests.

- Password protection: passwords are never stored in plaintext; bcrypt or an equivalent strong one-way password hashing scheme with per-user salt is required.
- Authorization: every history read/delete operation is filtered by the authenticated user identity. Guest runs are isolated from registered-user history.
- Upload security: reject non-CSV files, enforce the maximum size, validate MIME/type and structure, sanitize parsed values, and avoid executing uploaded content.
- Transport security: HTTPS/TLS 1.2+ is enforced in the deployed environment.
- Abuse prevention: rate limiting is applied to registration and login endpoints.
- Logging: logs contain request IDs, status, timing, and operational errors but exclude passwords, tokens, and sensitive dataset contents.
- Threat model mapping: spoofing → authentication; tampering → input validation and integrity controls; information disclosure → TLS and authorization; denial of service → limits/rate limiting; elevation of privilege → role checks.

4. Design

4.1 Design Overview

The runtime workflow is intentionally linear for usability: Upload → Configure → Evaluate → Results. Each stage produces a validated representation for the next stage. The backend uses service-level validation even when the frontend has already validated input, because client-side checks cannot be trusted.

- Upload stage creates a temporary dataset reference and returns a preview.
- Configuration stage creates a normalized evaluation request containing target column, selected features, algorithms, split ratio, and missing-value handling.
- Evaluation stage trains each selected algorithm on the training partition and computes metrics on the test partition.
- Results stage returns per-algorithm metrics, confusion matrices where applicable, ROC-AUC where applicable, comparison data, and a ranking summary.
- Persistence is performed only for authenticated users, with the run and its results associated with the authenticated user ID.

4.2 UML Sequence Diagrams

UML Sequence Diagram — Dataset Upload and Evaluation

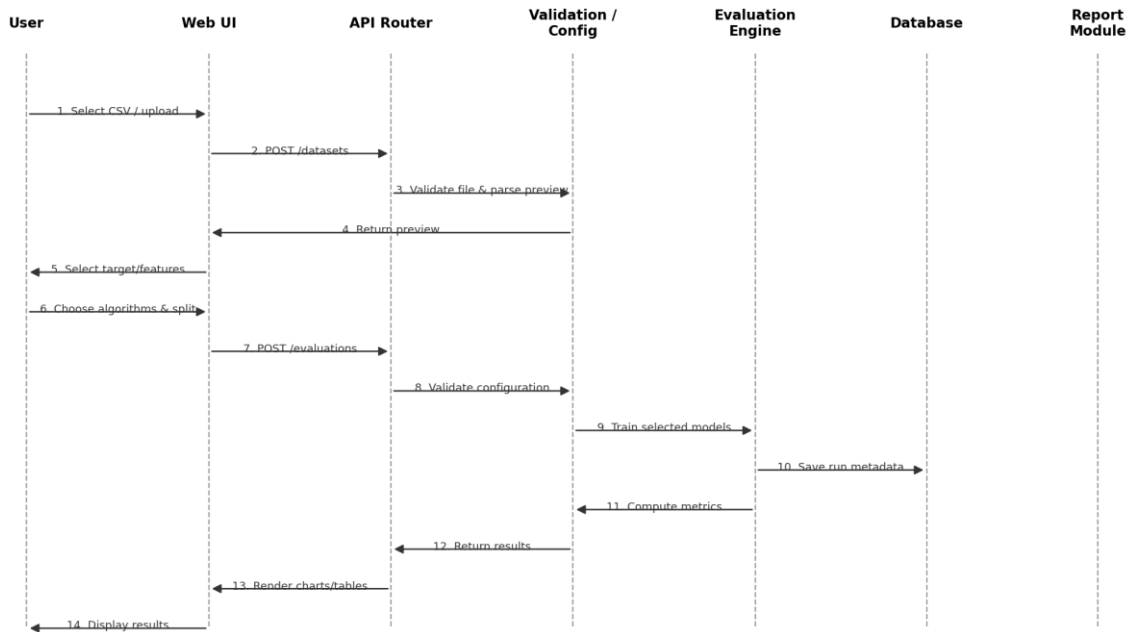


Figure 2. Sequence for dataset upload, configuration, model evaluation, and results display.

This sequence shows the primary evaluation flow. The API Router delegates validation and configuration checks before invoking the Evaluation Engine. Results are returned to the UI after metrics are computed; authenticated runs also persist metadata and results.

UML Sequence Diagram — View and Export History

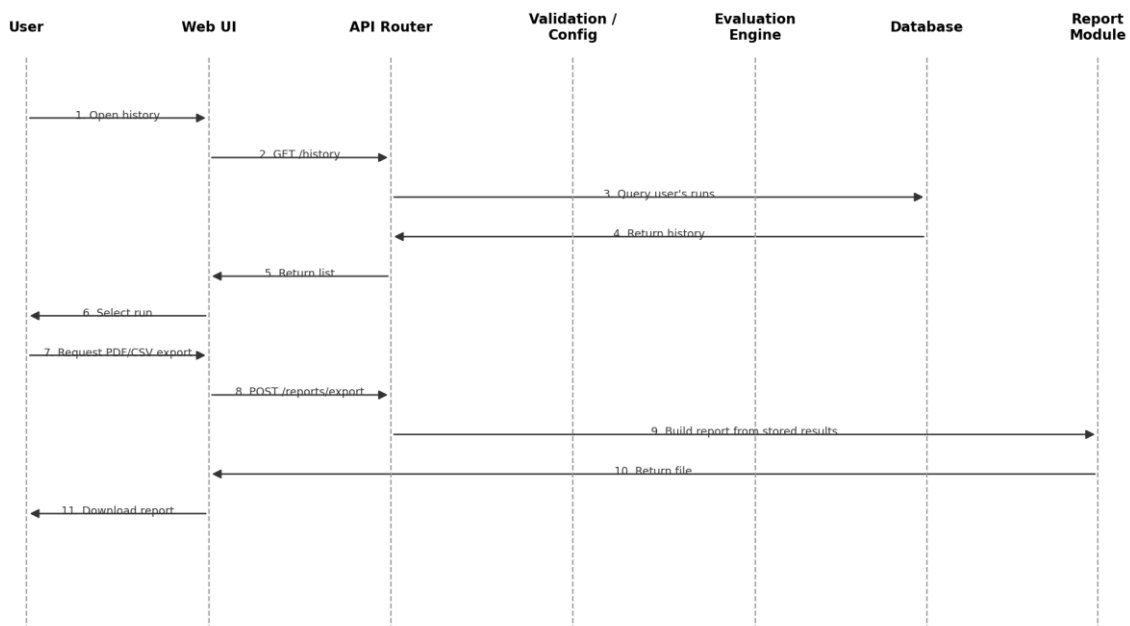


Figure 3. Sequence for viewing evaluation history and exporting a report.

The history flow demonstrates owner-scoped retrieval and report generation. The API must derive the user identity from the authenticated token/session rather than accepting an arbitrary user ID from the client.

4.3 API Design

The following REST endpoints represent the minimum service contract. Exact route naming may be adjusted during implementation, but request/response semantics should remain stable.

Endpoint	Method	Authentication	Purpose	Representative Response
/auth/register	POST	No	Create a user account	201 {user_id, message}
/auth/login	POST	No	Verify credentials and issue token/session	200 {access_token, user}
/auth/logout	POST	Yes	Invalidate session/token	204 or 200 {message}
/datasets/upload	POST	Optional	Upload and validate CSV; return preview	200 {dataset_id, columns, preview, warnings}
/evaluations	POST	Optional	Run selected algorithms with configuration	202/200 {run_id, status, results}
/evaluations/{run_id}	GET	Owner/guest scope	Retrieve results for a run	200 {run, algorithm_results, comparison}
/history	GET	Yes	List authenticated user's past runs	200 {items:[...]}
/history/{run_id}	DELETE	Yes	Delete an owned history entry	204 or 200 {message}
/reports/{run_id}/pdf	GET	Owner/guest scope	Export a PDF report	200 application/pdf
/reports/{run_id}/csv	GET	Owner/guest scope	Export raw metric results	200 text/csv

4.3.1 Example Evaluation Request

```
POST /evaluations
Content-Type: application/json
Authorization: Bearer <token> # omitted for guest runs
```

```
{
  "dataset_id": "ds_001",
  "target_column": "label",
  "feature_columns": ["age", "income", "score"],
  "algorithms": ["logistic_regression", "random_forest"],
  "test_size": 0.20,
  "missing_value_strategy": "median_imputation"
}
```

4.3.2 Example Evaluation Response

```
{
  "run_id": "run_001",
  "status": "completed",
  "results": [
    {
      "algorithm": "random_forest",
      "accuracy": 0.92,
      "precision": 0.91,
      "recall": 0.90,
      "f1_score": 0.905,
      "roc_auc": 0.95,
      "confusion_matrix": [[45, 5], [4, 46]]
    }
  ]
}
```

4.4 Error Handling, Logging and Monitoring

All API errors use standard HTTP status codes and a consistent JSON envelope:

```
{
  "error": {
    "code": "INVALID_DATASET",
    "message": "The uploaded file is not a valid CSV dataset.",
    "request_id": "req_123"
  }
}
```

- 400 Bad Request: invalid configuration, missing required fields, invalid split ratio, or malformed JSON.
- 401 Unauthorized: missing or invalid authentication token/session.
- 403 Forbidden: authenticated user attempts to access another user's history or artifact.
- 404 Not Found: unknown dataset, run, history entry, or artifact.
- 413 Payload Too Large: upload exceeds the configured maximum size.
- 415 Unsupported Media Type: non-CSV upload or unsupported content type.
- 422 Unprocessable Entity: CSV parses but fails schema/data validation.
- 429 Too Many Requests: authentication rate limit exceeded.
- 500/503: unexpected internal failure or temporary service unavailability; response must not expose stack traces or secrets.

Operational logs should capture request ID, endpoint, user scope identifier where appropriate, duration, status code, algorithm count, row/column counts, and failure category. Monitoring should include request latency, evaluation failure rate, queue/concurrency utilization, upload rejection counts, authentication failures, and database/storage errors.

4.5 UX Design

- Use a clear four-step flow: Upload, Configure, Evaluate, Results.
- Display a dataset preview containing the first N rows and column names before configuration.
- Make target-column selection visibly distinct from feature-column selection.
- Require at least one algorithm and reject invalid train-test split ratios before evaluation.
- Show missing-value handling explicitly, including whether imputation or row exclusion was applied.
- Display metrics in a per-algorithm table and a comparison chart; provide a confusion-matrix heatmap/grid for classification.
- Use accessible contrast, readable labels, keyboard-operable controls, and responsive layouts for desktop, laptop, and tablet browsers.
- Show progress or a clear busy state during evaluation and provide actionable, non-technical error messages.

4.6 Open Issues and Next Steps

- Finalize the frontend framework and charting library.
- Select FastAPI versus Flask and define the final project package structure.
- Define exact CSV schema rules, categorical-data handling, preprocessing defaults, and supported classification assumptions.
- Decide whether guest datasets/results use short-lived server storage, signed references, or in-memory scope.
- Implement and validate the optional admin usage-statistics/log view.
- Create automated unit tests for metric calculations against known reference datasets.
- Perform concurrency testing against the SRS target of at least 20 concurrent evaluation requests.
- Prepare deployment configuration with HTTPS, environment-based secrets, database migrations, and cleanup of temporary files.

5. Appendices

5.1 Glossary

- ML: Machine Learning
- CSV: Comma-Separated Values
- API: Application Programming Interface
- UI: User Interface
- F1-score: Harmonic mean of precision and recall
- ROC-AUC: Receiver Operating Characteristic—Area Under Curve
- JWT: JSON Web Token
- RTM: Requirements Traceability Matrix
- SADS: Software Architecture and Design Specification

5.2 References

- Project SRS: Web Based ML Model Evaluator, Version 1.0.
- IEEE 42010 concepts for architecture descriptions.
- OWASP secure application design and API security guidance.
- NIST SP 800-160 concepts for trustworthy secure system design.

5.3 Tools

- draw.io-style UML diagrams exported as PNG and embedded in this document.
- Python, FastAPI/Flask, pandas, NumPy, scikit-learn, and SQLite/PostgreSQL as proposed implementation technologies.
- A source-control system such as Git for versioning and review.

End of Software Architecture and Design Specification